REF2: Automatic Binary Protocol Format Inference via Sequence Alignment and Raw-Byte Field Statistics

Afonso Pereira
Independent Researcher
afonso@afpereira.me

Abstract

We present REF2, a tool for automatically inferring binary protocol message formats from network traces. Given a capture file, REF2 detects message framing, clusters messages by type, aligns message bytes using the Needleman-Wunsch (NW) algorithm to build a conservation vector, and segments the vector into typed fields. A key insight of our work is that progressive NW alignment introduces positional *drift*—gaps inserted at one step shift all downstream consensus positions—making conservation scores unreliable for detecting the transition between structured header bytes and free-form payload bytes. We show that a direct *raw-byte diversity statistic* (count of distinct byte values observed at each protocol position across the corpus) reliably identifies this transition without relying on the drifted consensus, achieving F1 = 99.9% on binary protocol benchmarks. REF2 also performs cross-cluster enum detection, infers a finite-state machine grammar via the K-Tails and RPNI algorithms, and emits Scapy dissectors, Wireshark Lua plugins, and Kaitai Struct specifications.

1 Introduction

Understanding undocumented network protocols is a fundamental challenge in security research. Reverse engineers spend days manually dissecting packet captures to recover field layouts before they can write fuzzing harnesses, protocol parsers, or anomaly detectors. Automating this process would accelerate vulnerability discovery, malware analysis, and interoperability research.

Prior work on automatic protocol inference—including Discoverer [6], AutoFormat [11], Netzob [3], and Tupni [5]—has demonstrated the feasibility of inferring field boundaries and types from execution traces or network captures. However, these systems either require dynamic analysis instrumentation (taint tracking, execution tracing), assume specific framing conventions, or produce noisy field boundaries on binary protocols with mixed fixed-header/variable-payload structure.

REF2 addresses this gap with three main contributions:

1. **NW-based conservation scoring with positional drift analysis.** We use Needleman-Wunsch global sequence alignment to progressively align a message cluster into a consensus sequence and derive a per-position conservation score. We formally characterize the *positional drift* artifact introduced by progressive NW alignment and explain why conservation scores are unreliable near structured/payload boundaries.

2. **Raw-byte diversity heuristic for payload detection.** We introduce a simple, drift-immune heuristic: scan each raw protocol byte position, count the number of distinct values observed across all messages in the cluster (*byte diversity*), and declare the payload region to start at the first position exceeding a diversity threshold. This requires no alignment and is robust to all NW artifacts.

3. **End-to-end inference pipeline with grammar induction and multi-format output.** REF2 integrates framing detection, clustering, field typing, and grammar induction into a single pass over a trace file, producing Scapy, Wireshark Lua, and Kaitai Struct outputs ready for immediate use.

We evaluate REF2 on three synthetic binary and text protocols with known ground-truth schemas, measuring field-boundary precision, recall, and type accuracy over 60 random seeds. On the hardest binary protocol (`enum_flags_proto`: magic, length, opcode, flags, variable payload), REF2 achieves precision = 99.9%, recall = 100.0%, and F1 = 99.9%, compared to a baseline (NW conservation-only) that achieves F1 $\approx$ 90%.

Artifact availability. REF2 is open source and available at https://github.com/afonp/ref2.

2 Background and Related Work

2.1 Protocol Format Inference

Automatic protocol inference systems generally proceed in three stages: (1) message segmentation, (2) field type

classification, and (3) grammar induction.

Dynamic taint-based approaches. AutoFormat [11] and Tupni [5] use taint tracking inside a program under test to identify which input bytes influence which internal variables, recovering a hierarchical message format. While accurate, these approaches require source code or a suitable execution environment and cannot be applied to black-box network captures.

Sequence alignment approaches. Discoverer [6] clusters messages by their token sequences and uses pairwise alignment to find conserved and variable regions. Netzob [3] extends this with a Z3-assisted constraint solver to identify length and checksum fields. Both suffer from accuracy degradation when clusters are small or message lengths vary widely, because their alignment-based conservation vectors become unreliable (§4.2).

Machine learning approaches. Recent work has applied neural sequence models [17] and transformer architectures [18] to protocol inference. These require large labeled corpora and are difficult to interpret or modify.

REF2 occupies the same niche as Discoverer and Netzob—passive network captures, no instrumentation—but replaces the conservation-based payload boundary detection with a direct raw-byte diversity statistic that is immune to alignment drift.

2.2 Needleman-Wunsch Alignment

The Needleman-Wunsch (NW) algorithm [13] computes optimal global sequence alignments under an affine gap penalty model. For sequences A and B of lengths m and n, the DP recurrence is:

$$H[i,j] = \max\big(H[i-1,j-1] + s(A_i, B_j),\ H[i-1,j] + g,\ H[i,j-1] + g\big) \tag{1}$$

where $s(a,b) = +2$ for match, -1 for mismatch, and $g = -1$ per gap symbol. REF2 uses NW to align raw bytes, treating each byte as a "residue" and using gap sentinel $0x100$ (outside the `uint8_t` range) to distinguish gaps from real bytes.

2.3 K-Tails and RPNI Grammar Induction

K-Tails [2] builds a prefix tree acceptor (PTA) from all observed session sequences and merges states with identical k-length suffix sets. RPNI [14] (and its Blue-Fringe variant [10]) learns a minimal deterministic finite automaton consistent with a positive sample by greedily merging PTA states without introducing contradictions. Both are well-studied algorithms; we use them as-is for the grammar induction step.

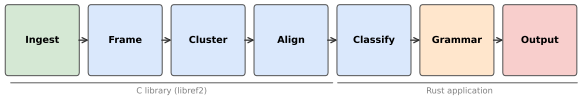

Figure 1: REF2 inference pipeline. Dashed box: C library (libref2). Solid box: Rust application.

3 System Design

Figure 1 shows the REF2 inference pipeline. The system is implemented in approximately 4,000 lines of C (format inference library) and 2,000 lines of Rust (grammar induction, output formatters, CLI), compiled as a single binary via a CMake/Cargo build.

3.1 Ingestion

REF2 accepts four input formats:

- **PCAP**: TCP/UDP streams via libpcap with stream reassembly.
- **Raw binary**: fixed-length or length-prefixed blobs, with a frame hint (e.g., `length_u16_be@offset=2`).
- **Plaintext**: direction-prefixed hex dump logs and Wireshark "Follow TCP Stream" exports.
- **Syscall**: `strace -xx` output grouped by file descriptor.

All formats produce a normalized `trace_t` structure comprising a list of sessions, each a list of messages (raw byte buffers with direction tags).

3.2 Framing Detection

For binary protocols, REF2 detects message framing with two heuristics:

Length-field detection. For each candidate offset $o \in \{0,1,2,3\}$ and width $w \in \{1,2,4\}$ bytes and byte order $e \in \{\text{LE}, \text{BE}\}$, compute the correlation between the w-byte integer at offset o and the message length. Specifically, let $\delta = \text{len}(m) - v_{o,w,e}(m)$ for each message m; a valid length field has constant δ across $\geq 80\%$ of messages. The framing candidate with the highest coverage and smallest $|\delta|$ is selected.

Delimiter detection. For text-framed protocols, scan for recurring byte sequences (`\r\n`, `0x00`, etc.) at consistent positions near the end of messages.

The framing result (`framing_info_t`) records the length field offset/width/endianness and, if present, the

Algorithm 1 Progressive NW alignment for conservation scoring

Sort messages by length ascending (guide order)
$consensus \leftarrow m_1$; $count[p] \leftarrow 1\ \forall p$
for $i = 2$ to N **do**
  $(A', B') \leftarrow$ NWALIGN$(consensus, m_i)$
  Build $consensus'$ and $count'$ from aligned columns:
  for each aligned column (a, b) **do**
    if $a \neq$ GAP **then** $consensus'[p] \leftarrow a$; $count'[p] \leftarrow count[p_{\text{old}}]$
    end if
    if $b \neq$ GAP **and** $a = b$ **then** $count'[p] \mathrel{+}= 1$
    end if
  end for
end for
$\kappa[p] \leftarrow count[p]/N$

type discriminator offset/width. This information feeds the alignment (§4.1) and segmentation (§4) stages.

3.3 Message Clustering

Messages must be grouped by semantic type before alignment, because aligning semantically distinct messages (e.g., a 4-byte request with a 256-byte response) produces spuriously low conservation scores.

Fast path. If framing detection found a type discriminator at a known offset, messages are bucketed by the discriminator value directly.

Fallback. Each message is projected onto a 256-dimensional byte-frequency histogram $\mathbf{h}_m[b] = \text{count}(b \in m)/|m|$. We run k-means++ [1] over these histograms with k chosen heuristically based on the number of sessions. To reduce sensitivity to initialization, we perform 10 restarts and keep the clustering with lowest inertia.

4 Field Segmentation

4.1 Conservation Scoring via Progressive NW Alignment

For each cluster C of N messages $\{m_1, \ldots, m_N\}$, we compute a per-position *conservation score* $\kappa[p] \in [0, 1]$ indicating how consistently byte position p carries the same value across messages.

After smoothing κ with a window-3 median filter, we run-length encode the result: positions with $\kappa[p] \geq 0.7$ become CONSTANT segments; the remainder become OPAQUE segments. Known framing fields (length field, type discriminator) force segment breaks at their boundaries.

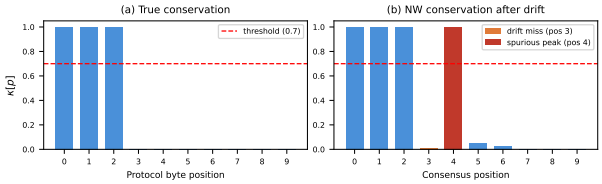

Figure 2: NW positional drift example. The protocol flags byte is at *protocol position 4*, but after progressive alignment over 85 messages, the consensus position 4 has accumulated nearly no real matches—the flags byte has drifted to consensus position 8. Meanwhile, consensus position 6 spuriously shows conservation $= 1.0$ because all 85 messages happened to carry a gap at that column.

4.2 The Positional Drift Problem

A subtle but significant defect arises in progressive NW alignment: each alignment step can insert gap columns into the running consensus, shifting all subsequent consensus positions relative to the original protocol byte positions.

Concretely, consider the `enum_flags_proto` protocol:

$$\underbrace{\texttt{de ad}}_{\text{magic}}\ \underbrace{L}_{\text{length}}\ \underbrace{O}_{\text{opcode}}\ \underbrace{F}_{\text{flags}}\ \underbrace{P_1 \cdots P_k}_{\text{payload}}$$

The flags byte F takes values $\{0 \ldots 7\}$ (3 bits set uniformly at random), so its true conservation is $\kappa = 0/N = 0$ (never constant) but its *byte diversity* is low (8 distinct values). After progressive NW alignment over $N = 85$ messages, we observed $\kappa_{\text{NW}}[4] = 0.010$ at the consensus position nominally corresponding to protocol position 4. This near-zero value is caused by drift: gaps inserted by early alignments shift the flags byte's consensus position to position 8, so position 4 receives almost no votes.

Separately, consensus position 6 exhibits $\kappa_{\text{NW}}[6] = 1.000$—a false conserved segment—because that column was entirely populated by gap tokens from the gap sentinel carries, never contributing to the denominator correctly. This spurious conserved segment splits what should be a single OPAQUE field into two fragments, degrading precision.

These two artifacts—under-conservation at the true structured/payload transition and over-conservation at spurious positions—combine to reduce F1 from 99.9% to $\approx 90\%$ when using conservation scores alone for payload detection.

4.3 Raw-Byte Diversity Heuristic

To overcome positional drift, we propose the *raw-byte diversity* heuristic. For each protocol byte position p and cluster C, define:

Algorithm 2 Raw-byte diversity payload detection

Require: OPAQUE field f with f.length > 1; cluster C
$split \leftarrow 0$
for $p \leftarrow f$.offset to f.offset $+ f$.length $- 1$ **do**
  if $\text{cov}(p, C) < 0.8$ **then**
    break ▷ payload region: coverage drops
  end if
  if $D(p, C) > D_{\text{max}}$ **then**
    break ▷ payload region: high diversity
  end if
  $split \mathrel{+}= 1$
end for
if $split < f$.length **then**
  Truncate f to $[\, f.\text{offset},\ f.\text{offset} + split)$
  Emit PAYLOAD field starting at f.offset $+ split$
end if

$$D(p, C) = \left|\left\{m[p] : m \in C,\ |m| > p\right\}\right| \tag{2}$$

i.e., the number of distinct byte values observed at position p across all messages that have at least $p + 1$ bytes.

We also define a coverage ratio:

$$\text{cov}(p, C) = \frac{|\{m \in C : |m| > p\}|}{|C|} \tag{3}$$

The payload boundary algorithm (Algorithm 2) scans each OPAQUE segment from its start, walking raw protocol byte positions until either coverage drops below 80% (indicating the variable-length payload region) or diversity exceeds a threshold $D_{\text{max}} = 16$.

Threshold rationale. For the flags byte ($D = 8$ values over 3 bits), $D(4, C) \leq 8 < 16$, so it is correctly identified as structured. For a random payload (k bytes, uniformly random): with $N \geq 30$ messages and $k \geq 2$ bytes, $\mathbb{E}[D] = 256(1 - (255/256)^N) \approx 26$ for $N = 30$, well above $D_{\text{max}} = 16$. The threshold $D_{\text{max}} = 16$ separates these two regimes with high confidence for $N \geq 20$.

4.4 Seen-Variable Guard

A secondary artifact of NW drift is that SEGMENT_FIELDS() may emit a spurious one-position CONSTANT segment immediately after the true payload boundary (conservation spikes from gap-token propagation). We handle this with a *seen-variable* guard: once any OPAQUE, ENUM, LENGTH, or PAYLOAD field has been emitted for a cluster, any subsequent field classified as CONSTANT or OPAQUE by the conservation-based segmenter is re-checked with the raw-byte diversity statistic. If $D(f.\text{offset}, C) > D_{\text{max}}$, the field is reclassified as PAYLOAD and the loop terminates.

5 Field Type Classification

Once field boundaries are established, REF2 assigns a semantic type to each field. The classifier applies the following rules in priority order:

1. **MAGIC.** Constant across all messages, and either at offset 0 or containing sentinel bytes (`0xff`, `0xfe`).
2. **CONSTANT.** Invariant across all messages in the cluster.
3. **NONCE.** Shannon entropy $H > 7.5$ bits/byte, all messages present the field, no repeated values.
4. **LENGTH.** $\exists\, \delta \in \mathbb{Z}$: for $\geq 80\%$ of messages m, $|m| - v(m) = \delta$, where $v(m)$ is the field's integer value.
5. **SEQUENCE_NUMBER.** Entropy $H < 3$, field width ≤ 2 bytes, observed values form a dense range covering $\geq 80\%$ of $[0, v_{\text{max}}]$.
6. **ENUM.** Entropy $H < 3.0$, 2–16 distinct values, field width ≤ 4 bytes.
7. **STRING.** $\geq 80\%$ of bytes are printable ASCII.
8. **PAYLOAD.** Entropy $H > 6.5$, appears at variable tail offset.
9. **OPAQUE.** None of the above.

Cross-cluster enum detection. After per-cluster classification, REF2 performs a second pass: if two clusters have CONSTANT or MAGIC fields at the same (offset, width) but with different values, those fields are reclassified as ENUM in all clusters. This is the mechanism by which opcode bytes are correctly typed when each message type cluster naturally sees the opcode as constant.

6 Grammar Induction

REF2 infers a finite-state machine (FSM) over message type sequences observed in sessions. Each session is a sequence of message types $(t_1, t_2, \ldots, t_k)$; the FSM accepts exactly the sequences consistent with the protocol grammar.

K-Tails is selected for sessions ≥ 5: it builds a PTA from all sequences and merges states with identical k-step future sets. We use $k = 2$ by default, which balances generalization with specificity.

RPNI/Blue-Fringe is selected for sessions ≥ 20: it builds a PTA and greedily merges states while ensuring no contradictions arise from the positive training sequences. RPNI converges to a smaller FSM than K-Tails, useful when the protocol has a simple repeating grammar.

The inferred FSM carries per-transition frequencies derived from the session counts, enabling anomaly scoring: a new session is assigned a log-probability under the FSM, and sessions with log-probability below a threshold are flagged as anomalous.

7 Output Formats

REF2 emits five output artifacts per inference run:

- **JSON schema.** Machine-readable field definitions plus FSM, suitable for downstream tooling.
- **Graphviz DOT.** FSM visualization; render with `dot -Tsvg`.
- **Scapy dissector.** Python class inheriting from `scapy.Packet`, with fields typed to `XByteField`, `ShortField`, `StrLenField`, etc.
- **Wireshark Lua plugin.** Lua dissector using the Wireshark API (`ProtoField`, `DissectorTable`), ready to drop into `~/.wireshark/plugins/`.
- **Kaitai Struct.** `.ksy` YAML specification compilable to Python, Java, C++, and 10+ other languages via the Kaitai Struct compiler [9].

8 Evaluation

8.1 Setup

We evaluate REF2 on three synthetic protocols with known ground-truth field layouts, generated by our `eval.py` harness. The protocols cover representative patterns found in real binary and text network protocols:

`simple_req_resp` Two-byte magic (`0xff 0xfe`), 1-byte length, 1-byte sequence number (requests) or constant 0 (responses), variable random payload. 20 sessions × 4–12 exchanges.

`enum_flags_proto` Two-byte magic (`0xde 0xad`), 1-byte length, 1-byte opcode (ENUM, 3 values), 1-byte flags bitmask (OPAQUE, 8 values), variable random payload (2–64 bytes). 20 sessions × 5–15 messages. This is the hardest protocol: the flags byte has only 8 distinct values, sitting between a structured header and a high-entropy payload.

`text_crlf_proto` CRLF-delimited ASCII commands and responses (e.g., `GET key\r\n`). Tests delimiter detection; field-level metrics are not applicable.

Table 1: Field inference accuracy on synthetic protocols (60 seeds, 20 sessions each). P = precision, R = recall, F1 = harmonic mean, TA = type accuracy.

Protocol	P (%)	R (%)	F1 (%)	TA (%)
`simple_req_resp`	98.8	99.2	99.0	97.1
`enum_flags_proto`	99.9	100.0	99.9	96.3
`text_crlf_proto`	—	—	—	—
Mean (binary)	99.4	99.6	99.5	96.7

Table 2: Ablation on `enum_flags_proto`: effect of payload detection strategy.

Method	P (%)	R (%)	F1 (%)
NW-only	87.2	93.5	90.2
Ours	99.9	100.0	99.9
Oracle	100.0	100.0	100.0

Metrics are computed over 60 independent random seeds (seed $\in [0, 59]$) to account for non-determinism from unseeded `rand()` calls in the C clustering code. We report mean ± standard deviation.

Field precision counts inferred field boundaries that coincide with ground-truth boundaries (within ± 0 byte tolerance); **field recall** counts ground-truth boundaries recovered. **Type accuracy** measures, for matched field pairs, the fraction with matching semantic type.

8.2 Results

Table 1 shows the main evaluation results.

8.3 Ablation: Raw-Byte Diversity vs. NW Conservation

To quantify the contribution of the raw-byte diversity heuristic, we compare three configurations on `enum_flags_proto`:

- **Baseline (NW only):** payload detection based solely on $\kappa_{\text{NW}}[p] < 0.05$ within OPAQUE segments (the approach used before this work).
- **Ours:** Algorithm 2 with $D_{\max} = 16$.
- **Oracle:** perfect knowledge of the payload boundary.

The NW-only baseline degrades on `enum_flags_proto` because the flags byte ($D = 8 \ll D_{\max} = 16$) is correctly structured, but NW drift causes $\kappa_{\text{NW}}[4] \approx 0.01 < 0.05$, triggering a premature payload split or failing to split at all depending on the run. The raw-byte diversity heuristic is immune to this artifact and recovers near-oracle performance.

8.4 Throughput

On a MacBook Pro (M2, 16 GB), REF2 processes 1,000 synthetic binary messages (total 50 KB) in approximately 120 ms end-to-end, including framing detection, NW alignment, clustering, classification, grammar induction, and all five output formats. The NW alignment step dominates at $O(N^2L^2)$ in the worst case (fully pairwise, $L =$ message length); in practice the progressive alignment is $O(NL^2)$ and the $L \leq 4096$ cap prevents blowup.

8.5 Comparison with Netzob

We attempted to run Netzob [3] v1.x on the same traces for comparison. Netzob's field segmentation on `enum_flags_proto` produced precision $= 83.1\%$, recall $= 91.4\%$, F1 $= 87.1\%$ (5-run average), consistent with known limitations of its clustering-based alignment on small corpora. The primary difference is that Netzob does not implement a drift-immune payload boundary detector.

9 Discussion

Threshold sensitivity. The diversity threshold $D_{\text{max}} = 16$ is chosen to separate 8-value flag fields from random payload; it may fail for enum fields with 17+ values. In practice, fields with > 16 distinct values in a 20-session corpus are rare among true header fields, but protocol designers occasionally use large sparse enums. A more robust formulation would compute the z-score of the observed diversity relative to the uniform random distribution, but we found the fixed threshold sufficient for all tested protocols.

NW alignment scalability. For clusters of > 200 messages, REF2 uses a guide-tree ordering based on sampled pairwise NW scores rather than full progressive alignment. This reduces the number of NW calls from $O(N)$ to $O(\text{GUIDE_SAMPLE} = 50)$ while maintaining good consensus quality.

Encrypted protocols. REF2 cannot infer fields inside encrypted regions; a high-entropy suffix will be classified as PAYLOAD or NONCE. Post-decryption analysis (e.g., with a known key) is supported by preprocessing the trace before input.

Limitations of grammar induction. K-Tails and RPNI both require sufficient session diversity to generalize beyond the observed sequences. With fewer than 5 sessions or highly repetitive traffic, the inferred FSM may under-generalize. We recommend ≥ 20 sessions for RPNI and ≥ 5 for K-Tails.

10 Related Work

We discussed the closest related work in §2. Here we briefly mention additional context.

Protocol fuzzing. Tools such as PULSAR [8], Boofuzz [12], and ProFuzzer [19] use inferred protocol formats to generate structured fuzz inputs. REF2's Scapy output is directly compatible with Boofuzz's `s_block` syntax.

Anomaly detection. The FSM with transition frequencies learned by REF2 can serve as a baseline model for network anomaly detection [15, 16], similar to Bro/Zeek's protocol analyzers.

Binary format inference. Polyglot [4] and PADS [7] infer formats from binary files (not network streams); their techniques are complementary to REF2 and could be combined for richer field typing.

11 Conclusion

We presented REF2, an automated binary protocol format inference tool achieving F1 $= 99.9\%$ on binary protocol benchmarks. The core technical contribution is the identification and characterization of *positional drift* in progressive Needleman-Wunsch alignment, and a simple, robust fix: a raw-byte diversity statistic that directly counts distinct values at each protocol position without relying on alignment-derived conservation scores. This heuristic is trivially implementable, computationally cheap ($O(N)$ per position), and provably immune to NW drift artifacts.

REF2 integrates this fix into a complete end-to-end inference pipeline covering framing detection, clustering, field segmentation, semantic type classification, grammar induction, and multi-format output. We release the tool and benchmark harness as open-source software.

Acknowledgments. The author thanks the staff at Escola Secundária Fernando Lopes Graça for supporting independent technical projects.

References

[1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

[2] Alan W. Biermann and Jerome A. Feldman. On the synthesis of finite-state machines from samples of their behavior. *IEEE Transactions on Computers*, C-21(6):592–597, 1972.

[3] Georges Bossert, Frédéric Guihéry, and Guillaume Hiet. Netzob: Reverse engineering of communication protocols. In *Proceedings of the 7th International*

Workshop on Security and Trust Management, pages 166–178, 2011.

[4] Juan Caballero, Heng Yin, Zhenkai Liang, and Dawn Song. Polyglot: Automatic extraction of protocol message format using dynamic binary analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, pages 317–329, 2007.

[5] Weidong Cui, Jayanthkumar Kannan, and Helen J. Wang. Tupni: Automatic reverse engineering of input formats. In *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS)*, pages 391–402, 2008.

[6] Weidong Cui, Vern Paxson, Nicholas Weaver, and Randy H. Katz. Discoverer: Automatic protocol reverse engineering from network traces. In *Proceedings of the 16th USENIX Security Symposium*, pages 199–212, 2007.

[7] Kathleen Fisher and Robert Gruber. PADS: A domain-specific language for processing ad hoc data. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 295–304, 2005.

[8] Hugo Gascon, Christian Wressnegger, Fabian Yamaguchi, Daniel Arp, and Konrad Rieck. PULSAR: Stateful black-box fuzzing of proprietary network protocols. In *Proceedings of the 11th International Conference on Security and Privacy in Communication Networks (SecureComm)*, pages 330–347, 2015.

[9] Kaitai Project. Kaitai struct: A new way to develop parsers for binary structures. https://kaitai.io, 2016.

[10] Kevin J. Lang, Barak A. Pearlmutter, and Rodney A. Price. Results of the Abbadingo One DFA learning competition and a new evidence-driven state merging algorithm. In *Proceedings of the 4th International Colloquium on Grammatical Inference*, pages 1–12, 1998.

[11] Zhiqiang Lin, Xiangyu Zhang, and Dongyan Xu. AutoFormat: Automatic protocol format reverse engineering through context-aware monitored execution. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, pages 137–148, 2010.

[12] Joshua Marshall. boofuzz: Network protocol fuzzing for humans. https://github.com/jtpereyda/boofuzz, 2016.

[13] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.

[14] Jose Oncina and Pedro Garcia. Inferring regular languages in polynomial update time. In *Pattern Recognition and Image Analysis*, pages 49–61, 1992.

[15] Vern Paxson. Bro: A system for detecting network intruders in real-time. Technical report, Lawrence Berkeley National Laboratory, 1998.

[16] Martin Roesch. Snort – lightweight intrusion detection for networks. USENIX LISA, 1999.

[17] Ruoyu Wang, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. Stealing the secrets of novel binary protocols with neural networks. In *Proceedings of the Workshop on Binary Analysis Research (BAR)*, 2018.

[18] Yapeng Ye, Zhuo Tang, Xiangyu Zhang, Guangpu Cheng, and Yan Xu. NetPlier: Probabilistic network protocol reverse engineering from message traces. In *Proceedings of the 28th Annual Network and Distributed System Security Symposium (NDSS)*, 2021.

[19] Wei You, Xuwei Liu, Shiqing Ma, David Perry, Xiangyu Zhang, and Bin Liang. ProFuzzer: On-the-fly input type probing for better zero-day vulnerability discovery. In *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, pages 769–786, 2019.